Proofbox

Containment-first agent execution. Evidence-backed action.

Open nothing unsafely.
Believe nothing unproven.

Proofbox inspects suspicious links, verifies bills, and checks letter requirements using isolated execution, deterministic checks, and a private evidence ledger.



Built for Vultr Agent Rush / Blast Radius Zero. The separate demo shows the product in use.

The problem is not a lack of answers. It is a lack of proof.

People must assess risky inputs and consequential paperwork without exposing themselves or losing context.

WHAT PEOPLE FACE

Suspicious links

Messages can lead to credential requests, misleading pages, redirects, or unwanted downloads.

Disputed bills and official letters

Charges, readings, requirements, and response dates need checking against explicit rules.

Scattered evidence

Documents, findings, replies, and deadlines are hard to assemble into one reviewable record.

WHO THE SYSTEM SERVES

Citizens

A guided way to understand findings and retain the artifacts behind them.

Support and advice teams

A consistent case file for reviewing work without reconstructing a conversation.

Service operators

A reusable pattern for controlled intake and evidence-backed workflow automation.

Target value: clearer decisions, less repeated checking, and a reliable handoff between people.

Three inspection lenses. One private case file.

Each lens has a bounded task and an explicit output; the case file connects them.

SURFACE	INPUT AND PROCESSING	OUTPUT
Link Lens	HTTP(S) URL; isolated browser observation and static indicators.	Verdict, reasons, trace artifacts, and containment events.
Bill Lens	Bill PDF; optional reading history and outage context; tariff recomputation.	Findings, calculation receipts, and a draft dispute letter.
Letter Lens	Official letter plus supporting PDFs; requirement extraction and layout checks.	Checklist findings, explicit dates, and derived check facts.
Case File	Attached findings, confirmed dates, reply proposals, and case-owned receipts.	Evidence ledger, clocks, actions, and versioned PDF / ZIP packs.

Scope is inspection and preparation, not automatic form submission or legal representation.

From untrusted input to a reviewable outcome.

The workflow separates interpretation, execution, verification, and user-controlled state changes.

01 / INTAKE

Validate the input. Create a job and preserve its context.

→

02 / EXECUTE

Run a browser or PDF/code task in the worker sandbox.

→

03 / VERIFY

Apply static checks, arithmetic, schemas, and domain rules.

→

04 / EXPLAIN

Return findings, caveats, and receipt references.

→

05 / PRESERVE

Attach to a case. Confirm dates. Export the record.

Job events describe execution; the case ledger preserves selected evidence and confirmed actions.

An interpretation is not proof. A completed job is not automatically a confirmed case action.

Observe a page without becoming its next victim.

The browser gathers behavior and artifacts; the person does not need to interact with the inspected page.

INSPECTION PIPELINE

Validate the URL

Accept absolute HTTP(S) URLs. Reject embedded credentials, local hosts, private IP literals, and invalid ports.

Run the browser in isolation

Playwright records redirects, visible text, form fields, requests, downloads, and an inspection image.

Collect behavioral evidence

Blocked requests and quarantined downloads become job events; artifacts return to the control plane.

DECISION LOGIC

Static indicators

Check domain/brand mismatch, sensitive inputs, redirect chains, punycode, raw IPs, and risk indicators.

Visual and trace interpretation

A vision-role model interprets the captured page alongside the trace. Deterministic danger rules override its result.

Explicit uncertainty

Results distinguish dangerous, suspicious, no issue found, and could not check. No issue found does not mean safe.

The browser observes; it does not fill or submit forms. Private-network protection also depends on egress policy.

Use language models for extraction, not arithmetic authority.

A billing finding must be traceable to the source fields, the rule used, and the calculation.

FROM PDF TO CHECKABLE FACTS

Isolated parsing

The uploaded bill is text-extracted in the worker sandbox. Preserve its filename, size, page count, and hash.

Structured extraction

The extractor produces typed bill fields. Unreadable inputs do not silently become successful checks.

Deterministic tariff checks

Decimal arithmetic recomputes charges. Reading history and outage context support additional checks when supplied.

FROM FINDING TO ACTION

Independent code path

An analyst-generated program executes in the no-network sandbox; its output is data to verify, not trusted control events.

Receipt-backed findings

Store billed, recomputed, and difference values with explanations and calculation artifacts.

Draft, not automatic dispute

Generate a reviewable dispute letter. Attaching a draft alone must not start a statutory response clock.

Municipal tariff and escalation rules remain unverified. Missing history means the related checks are skipped.

A correct calculation still needs a correct rule.

Illustrative arithmetic only: these values are not a product test result or an official tariff.

LAYER	ILLUSTRATIVE EXAMPLE	WHAT IT ESTABLISHES
Extracted facts	Usage: 12 units. Reported usage charge: R 220.00.	The fields to examine; extraction may still need correction.
Assumed rule	First 6 units at R 10.00; next 6 units at R 20.00.	The basis of recomputation, subject to source verification.
Recomputation	(6 x R 10.00) + (6 x R 20.00) = R 180.00.	A deterministic result under the stated assumptions.
Difference	R 220.00 - R 180.00 = R 40.00.	A discrepancy to review, not proof of unlawful billing.

The receipt must preserve the rule and inputs. Reproducible arithmetic cannot validate an unofficial rule.

Translate correspondence into requirements and check facts.

The official letter is interpreted; supporting documents are checked inside the isolated worker.

PROCESSING

Letter and supporting PDFs

Accept one official letter and up to four supporting PDFs, within the documented upload limits.

Parse before interpreting

Extract text inside the no-network sandbox. The model extracts requirements and explicit dates from the official letter.

Checklist execution

Layout-specific code checks supporting documents. Derived facts summarize presence, dates, and checklist outcomes.

PRIVACY AND LIMITS

Minimize model exposure

The supporting-document privacy boundary sends derived check facts rather than full supporting text or raw identity numbers.

Retain useful context

Requirement findings and explicit response or appointment dates can be attached to a private case.

Do not imply authentication

Checks are layout-based, not authenticity or identity verification. Scanned/image-only PDFs are unsupported.

The grant-review checklist is unverified. An extracted requirement is not an eligibility determination.

Preserve a ledger, not just a conversation.

The case file brings evidence, receipt ownership, deadlines, and correspondence into one workflow.

EVIDENCE LIFECYCLE

Attach completed findings

Link and document outcomes become evidence entries with case-specific sequence numbers and derived facts.

Stable references

Sequence allocation locks the case row; evidence numbers cannot be reassigned. Claims can refer to entries such as [E1].

Case-owned receipts

Receipt copies belong to the case. Deletion removes these copies rather than leaving orphaned case artifacts.

CONTROLLED CASE ACTIONS

Start clocks deliberately

A person confirms when a dispute was sent. Merely drafting or attaching correspondence is not enough.

Review replies before applying them

Model extraction creates a separate proposal. Confirmation is required before updating case state.

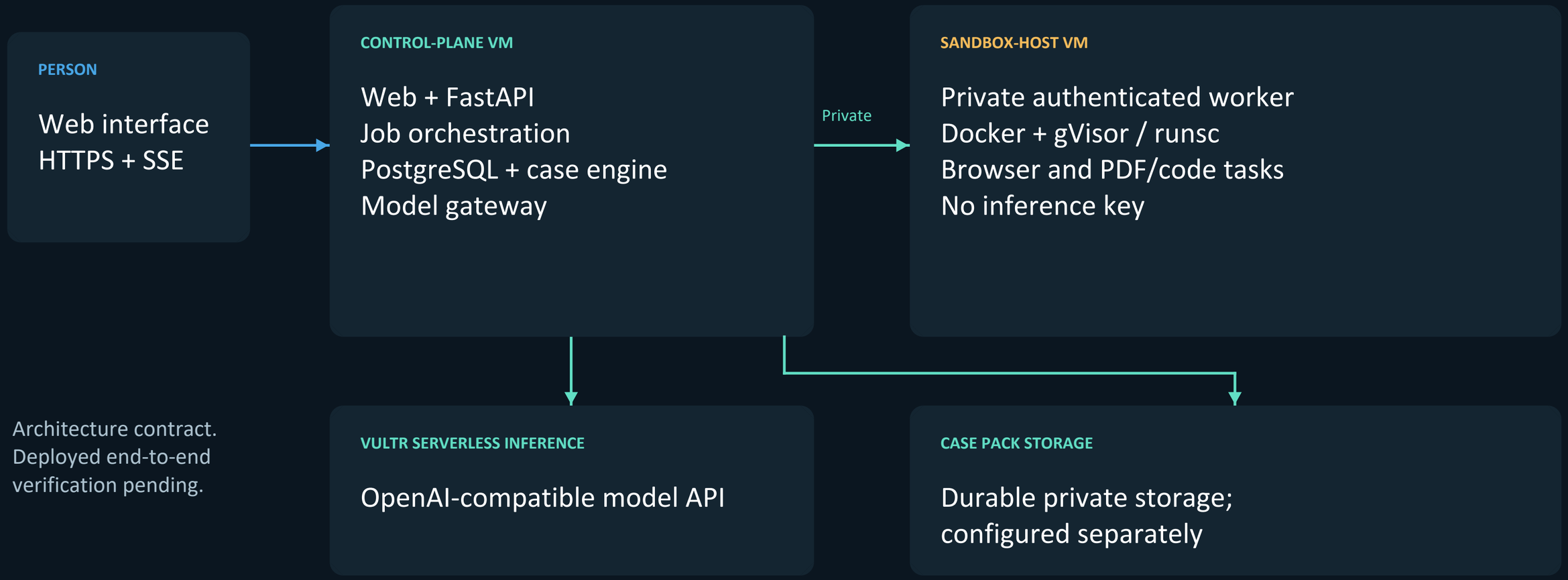
Export a dated snapshot

Pack generation preserves the selected evidence, claims, clock state, and receipt hashes in immutable versions.

Private access is possession-based: anyone with the exact token link can view the case. Share it with care.

Keep the authority outside the execution boundary.

Vultr hosts separated control-plane and sandbox tiers; model calls originate from the control plane.



Architecture contract.
Deployed end-to-end
verification pending.

Small, explicit responsibilities across the stack.

The implementation separates public orchestration, untrusted execution, model interpretation, and evidence export.

COMPONENT	RESPONSIBILITY	KEY BOUNDARY
Web application	Link, bill, letter, and case interfaces; displays streamed progress and findings.	No untrusted browser/code execution on the user's device.
FastAPI control plane	Validate intake, claim jobs, call the worker, evaluate results, and expose case APIs.	Owns inference access and consequential case-state changes.
Worker and sandbox	Run browser, parse_bill, parse_document, and code modes; collect output and lifecycle events.	Private API; per-task resource and execution restrictions.
PostgreSQL + pack builder	Persist jobs, events, model-call records, receipts, case state, and pack snapshots.	Integrity checks before export; durable pack storage required.

The model proposes or interprets. The API, deterministic logic, and user confirmation govern the workflow.

Assign models to bounded jobs, not unrestricted authority.

Current role mappings are visible in the gateway and configuration; no model quality ranking is claimed.

ROLE	CONFIGURED MODEL	RESPONSIBILITY
Router	deepseek-v4.1-flash-1017	Role mapping for classification and routing.
Extractor	qwen3.8-27b	Structured extraction from permitted source text.
Analyst / coder	deepseek-v4.1-flash-1017	Generate a program for isolated verification.
Writer	glm-5.3	Plain-language advice and reviewable draft letters.
Vision verifier	mimo-v2.6-pro	Interpret Link Lens page capture and trace.

Fallback models and translation are listed in YAML; gateway failover/translation must not be assumed implemented.

Treat external content as data, including model inputs.

The control plane centralizes model access, structured validation, privacy checks, and call accounting.

REQUEST AND RESPONSE

Delimit untrusted content

Serialize input as JSON and escape angle brackets before enclosing it in an untrusted-data boundary.

Validate structured output

A Pydantic schema validates JSON responses where a schema is required. The gateway strips code fences before validation.

Bound the retry loop

At most three attempts. Validation errors inform the next attempt; exhausted attempts return an explicit error payload.

CONTROL AND VISIBILITY

Centralize inference credentials

The Vultr inference key remains in the control-plane environment, not the browser or worker.

Apply the privacy gate

Check outgoing textual prompt content for identity-number patterns before submitting the request.

Record each call

Persist role, model, token counts, latency, and outcome. These support later operational measurement, not invented benchmarks.

Prompt boundaries reduce instruction confusion; they do not replace sandboxing, validation, or data-access policy.

Different threats need different enforcement layers.

Containment is a defense-in-depth design, not a claim that every possible escape has been eliminated.

THREAT	INTENDED ENFORCEMENT	RESIDUAL BOUNDARY
Hostile page or PDF	Separate worker; disposable execution; restricted container permissions.	Escape resistance must be tested on the deployed runtime.
Private-network access	No-network code mode; browser egress proxy and destination policy.	Validate DNS, redirects, metadata targets, and direct egress.
Prompt injection	Trusted parsing events, untrusted-data delimiters, schemas, and deterministic checks.	Not a proof of complete prompt-injection resistance.
Sensitive-data leakage	Prompt privacy gate, private case access, export exclusions, and integrity checks.	Pattern detection is not comprehensive personal-data detection.

Blast Radius Zero is the challenge objective. The presentation does not assert certified zero risk.

Enforce limits where the work actually executes.

The following values are current worker settings, not a measured throughput or performance guarantee.

CONTROL	BROWSER / LINK LENS	BILL / DOCUMENT / CODE
Runtime and network	gVisor / runsc; internal network with proxy.	gVisor / runsc by default; network none.
Memory / CPU / processes	1 GB; 1 CPU; 128 process limit.	512 MB; 0.5 CPU; 64 process limit.
Filesystem / privilege	Read-only root; dropped capabilities; no new privileges.	Read-only root and inputs; dropped capabilities; no new privileges.
Wall-clock limit	25 seconds for the browser worker task.	20 seconds by default; configurable.
Temporary data	Ephemeral output and download quarantine.	64 MB /tmp; isolated input directory.

Production must preserve these restrictions, confirm teardown, and never weaken runsc silently to make a task pass.

Show what happened, and what could not be established.

Job state and trusted worker events make execution inspectable without elevating model output into control signals.

LIFECYCLE AND DATA

Execution events

Record container start, artifact collection, prohibited requests, quarantined downloads, resource termination, and completion.

Worker-to-API streaming

Worker responses use NDJSON event records. The control plane persists job events and exposes SSE to the web interface.

Separate data from authority

Model-written code output is returned as data. Only trusted parser instruction_ignored records are forwarded as events.

FAILURE SEMANTICS

Unreadable or unavailable input

Parsing and inspection errors must be distinct from successful checks. Link Lens includes a could_not_check outcome.

Incomplete context

Missing bill history skips history-based checks instead of fabricating earlier readings.

Operational visibility

Failed jobs and model-call outcomes retain diagnostic context. A teardown event alone is not independent proof of cleanup.

Do not interpret a quiet trace, a reassuring summary, or a lifecycle label as proof that all checks passed.

Connect each claim to its inputs and artifacts.

The pack builder enforces reference validity and receipt integrity before producing the export.

01 / SOURCE

Preserve the input and its fingerprint.

→

02 / FINDING

Store derived facts and receipt references.

→

03 / LEDGER

Allocate stable case evidence numbers.

→

04 / EXPORT

Resolve claims and validate receipt hashes.

Claims such as [E1] must resolve to evidence in the pack. Receipt bytes are checked against recorded SHA-256 values. Invalid references or mismatched hashes stop pack generation.

A hash detects a content mismatch. It is not a digital signature, proof of authenticity, or proof of truth.

Deliver an evidence package another person can inspect.

The export is a versioned snapshot, not an automatically updated live document.

PDF EVIDENCE PACK

Case and version context

Identify the case, pack version, generation time, and applicable rule caveats.

Numbered claims and clocks

Link numbered claims to evidence sections; include due dates and clock state from the snapshot.

Evidence and drafted correspondence

List evidence entries and receipt hashes; include relevant draft dispute-letter content.

ZIP RECEIPT ARCHIVE

Reviewable artifacts

Package permitted receipt artifacts associated with the versioned evidence snapshot.

Supporting-document exclusion

Replace Letter Lens supporting PDFs with placeholders containing the source hash and privacy-protected metadata.

Integrity and privacy gates

Reject mismatched hashes and detected 13-digit identifiers. Store immutable PDF/ZIP versions under CASE_STORAGE_DIR.

Pack files need durable, private storage and consistent backup. Temporary container storage is insufficient.

Minimize exposure throughout the case lifecycle.

Controls reduce specific risks; they do not establish full POPIA compliance or comprehensive data-loss prevention.

BOUNDARY	CURRENT MECHANISM	IMPORTANT QUALIFICATION
Case access	Random token in URL fragment; API bearer header; database stores its SHA-256 hash.	Anyone possessing the link has access; sharing is a security decision.
Model prompts	Supporting-document check facts; textual identity-number pattern gate.	Pattern matching does not detect every sensitive field or image.
Evidence export	Supporting PDFs replaced with hash-bearing placeholders; ID-number gate.	Primary source and derived facts still require appropriate handling.
Retention and deletion	Case-owned receipts, expiry cleanup, and deletion workflow.	Verify deployed cleanup, backup retention, logs, and access policy.

Use synthetic data for the hackathon. Real-record handling requires a privacy and operational review.

Suggestions must not silently become consequential actions.

Explicit confirmation connects the case record to what a person actually did.

DEADLINE RULES

Anchor to a confirmed event

Municipal dispute clocks begin from the confirmed sent date, not the document-generation date.

Count business days in code

Counting starts on the next business day after the sent date; non-business days are skipped.

Expose rule uncertainty

Holiday dates, response periods, and escalation routes require verification against official sources.

REPLY PROPOSALS

Extract into a separate proposal

A reply can suggest changes without writing new evidence, clocks, or actions immediately.

Apply only after confirmation

The person reviews the proposal before the case state changes; attaching a draft does not start a dispute clock.

Discard deliberately

Discarding a proposal clears its raw reply text and extracted proposal. Reminders are derived from due dates.

A code-derived deadline is only as reliable as its anchor date, rule source, and holiday calendar.

Production readiness is an operational gate, not a slide claim.

Deploy the release in a controlled order and verify each boundary against the actual environment.

RELEASE PREPARATION

Pin and review the release

Record the intended commit and images. Review existing host changes and establish a maintenance and rollback owner.

Back up before migration

Back up PostgreSQL and pack storage consistently. Test restoration and review startup migrations on a database copy.

Rebuild sandbox images first

Ensure both bill and document parsers exist under runsc before updating the worker, API, and web services.

DEPLOYMENT CHECKS

Protect the private worker

Restrict network access and use a unique worker token. Do not expose the worker or Docker socket publicly.

Provision durable pack storage

Configure writable CASE_STORAGE_DIR, permissions, backup, and a consistent shared path if multiple API instances are used.

Verify the full workflow

Check all lenses, event streaming, case access, confirmation, pack downloads, privacy gates, and expiry cleanup.

An application-image rollback does not undo schema changes. Restore and migration procedures must be tested.

Useful implementation. Clearly defined remaining work.

Current source and deployment notes support workflow claims, not certified production or security outcomes.

STATUS	SCOPE	NEXT ACCEPTANCE EVIDENCE
Implemented	Three lenses; case API; clocks; reply confirmation; pack export; receipt and prompt privacy gates.	Release-specific regression and end-to-end results.
Unverified	Live deployment behavior, sandbox escape resistance, domain rules, holidays, and grant checklist.	Recorded deployment tests; official sources; adversarial test results.
Known limitations	Text PDFs only; layout-specific letter checks; possession-based case access.	Broader-format evaluation and a privacy/access review.
Next / proposed	Pilot workflow, measured accuracy and review effort, stronger operational observability.	Documented quality, cost, latency, and privacy acceptance thresholds.

No user-adoption, accuracy, cost-saving, latency, or escape-test numbers are claimed in this presentation.

Useful agent work, bounded execution, reviewable evidence.

The differentiator is the combination: interpretation alone cannot provide containment or a durable handoff.

WHAT MAKES THE SYSTEM DISTINCT

Execution-aware inspection

The link is observed in an isolated runtime rather than merely described from its URL.

Verification-aware interpretation

Models help read and explain; rules and deterministic calculations keep critical conclusions checkable.

Evidence-aware workflow

Findings survive as receipt-linked case entries and versioned exports, with human confirmation at consequential steps.

HOW VALUE SHOULD BE MEASURED

Review efficiency

Measure case preparation and reviewer reconstruction time against a manual baseline.

Finding quality

Evaluate extraction accuracy, false positives, missed findings, and agreement with verified domain rules.

Operational fitness

Measure task reliability, per-case inference cost, latency, cleanup success, and privacy-control outcomes.

Proofbox's promise: inspect the input, verify the claim, preserve the evidence, and keep the person in control.

A system presentation grounded in inspectable implementation.

Source paths are relative to sandbox-spike. Separate product demonstration is submitted alongside this document.

CLAIM AREA	IMPLEMENTATION / DOCUMENTATION ANCHOR
Orchestration and jobs	apps/api/main.py; apps/api/link_lens.py; apps/api/lenses/bill/job.py
Document workflows	apps/api/lenses/bill/pipeline.py; apps/api/lenses/letter_job.py
Inference and sandbox controls	apps/api/gateway.py; config/models.yaml; apps/worker/main.py; apps/worker/bill_modes.py
Case, integrity, and privacy	apps/api/casefile.py; apps/api/pack_builder.py; docs/decisions.md
Deployment and limitations	docs/deployment-guide.md; docs/decisions.md; apps/web/vultr.html

Open nothing unsafely. Believe nothing unproven. Preserve evidence that can be reviewed.